

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل دهم: Deep learning

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 10.1 Basic concepts
 - 10.1.1 What is deep learning?
 - 10.1.2 Backpropagation algorithm
 - 10.1.3 Key challenges for training deep learning models
 - 10.1.4 Overview of deep learning architecture
- 10.2 Improve training of deep learning models
 - 10.2.1 Responsive activation functions
 - 10.2.2 Adaptive learning rate
 - 10.2.3 Dropout
 - 10.2.4 Pretraining
 - 10.2.5 Cross-entropy
 - 10.2.6 Autoencoder: unsupervised deep learning
 - 10.2.7 Other techniques
- 10.3 Convolutional neural networks
 - 10.3.1 Introducing convolution operation
 - 10.3.2 Multidimensional convolution
 - 10.3.3 Convolutional layer
- 10.4 Recurrent neural networks
 - 10.4.1 Basic RNN models and applications
 - 10.4.2 Gated RNNs
 - 10.4.3 Other techniques for addressing long-term dependence
- 10.5 Graph neural networks
 - 10.5.1 Basic concepts
 - 10.5.2 Graph convolutional networks
 - 10.5.3 Other types of GNNs

Section 10.1

10.1.2. Backpropagation algorithm



پس انتشار (Backpropagation)

پس انتشار (Backpropagation) که مخفف "انتشار معکوس خطاها" است، یک الگوریتم کلیدی در آموزش شبکه‌های عصبی مصنوعی می‌باشد. این یک الگوریتم یادگیری نظارتی (supervised learning) است که وزن‌های اتصالات شبکه را برای حداقل کردن تفاوت بین خروجی پیش‌بینی شده و خروجی واقعی تنظیم می‌کند. الگوریتم پس انتشار شامل دو گام اصلی است:

- گام پیشرو (Forward Pass):** داده ورودی از طریق شبکه عصبی تغذیه می‌شود و شبکه خروجی پیش‌بینی شده را محاسبه می‌کند. فعال‌سازی هر لایه با استفاده از مجموع وزن‌دار ورودی‌ها محاسبه شده و از طریق یک تابع فعال‌سازی (activation function) عبور داده می‌شود.

- **گام پسرو (Backward Pass - Backpropagation):** خطا با مقایسه خروجی پیش‌بینی شده با خروجی واقعی با استفاده از یک تابع زیان (loss function) محاسبه می‌شود. سپس الگوریتم به صورت معکوس از طریق شبکه عمل می‌کند و وزن‌ها را برای حداقل کردن خطا تنظیم می‌نماید. این تنظیم با استفاده از گرادیان (gradient) تابع زیان نسبت به وزن‌ها، که از طریق قاعده زنجیره‌ای (chain rule) حساب دیفرانسیل محاسبه می‌شود، انجام می‌گردد.
- **بهینه‌سازی (Optimization):** یک الگوریتم بهینه‌سازی (مانند گرادیان کاهشی) اغلب برای به‌روزرسانی تکراری وزن‌ها در جهتی که خطا را حداقل می‌کند، به کار گرفته می‌شود.

پس‌انتشار به شبکه‌های عصبی اجازه می‌دهد از مثال‌ها یاد بگیرند و پارامترهای خود را برای بهبود عملکرد در یک وظیفه خاص تنظیم کنند. این یک مفهوم بنیادی در آموزش مدل‌های یادگیری عمیق است.

```
In [ ]: import numpy as np
from sklearn.datasets import make_classification
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# --- تولید داده‌ی مصنوعی برای مسئله‌ی طبقه‌بندی ---
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# --- تقسیم داده‌ها به مجموعه آموزش و آزمون ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- تعریف مدل شبکه عصبی ساده با یک لایه پنهان شامل ۵ نورون ---
model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, random_state=42)

# --- (Backpropagation) آموزش مدل با استفاده از الگوریتم پس‌انتشار خطا ---
model.fit(X_train, y_train)

# --- پیش‌بینی برچسب‌ها برای داده‌های آزمون ---
y_pred = model.predict(X_test)

# --- محاسبه دقت مدل ---
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

Accuracy: 0.82

توضیحات مثال کد پایتون

در این مثال، یک شبکه عصبی ساده با استفاده از کلاس **MLPClassifier** از کتابخانه **scikit-learn** برای حل یک مسئله‌ی طبقه‌بندی دودویی آموزش داده شده است:

- ابتدا داده‌های مصنوعی با ۲۰ ویژگی و ۲ کلاس تولید شده‌اند.
- داده‌ها به دو بخش آموزش (۸۰٪) و آزمون (۲۰٪) تقسیم شده‌اند.
- مدل شبکه عصبی شامل یک لایه پنهان با ۵ نورون تعریف شده و با الگوریتم **Backpropagation** آموزش دیده است.
- پس از آموزش، مدل روی داده‌های آزمون تست شده و دقت آن محاسبه شده است.

این مثال برای آموزش مفاهیم پایه‌ای شبکه‌های عصبی در محیط‌های سبک بسیار مناسب است. برای مسائل پیچیده‌تر، استفاده از کتابخانه‌هایی مانند **TensorFlow** یا **PyTorch** توصیه می‌شود.

Section 10.2

10.2.1. Responsive activation functions

توابع فعال‌سازی واکنشی (Responsive Activation Functions)

توابع فعال‌سازی (Activation functions) با وارد کردن ناخطی‌بودن (non-linearity) به شبکه، نقش حیاتی در مدل‌های یادگیری عمیق ایفا می‌کنند. توابع فعال‌سازی واکنشی (Responsive) برای بهبود فرآیند آموزش با رسیدگی به مسائلی مانند گرادیان‌های ناپدیدشونده (vanishing gradients) و قادر ساختن مدل به یادگیری مؤثرتر طراحی شده‌اند. برخی از توابع فعال‌سازی واکنشی محبوب عبارتند از:

- **ReLU (Rectified Linear Unit):** ReLU به دلیل سادگی و اثربخشی آن به طور گسترده‌ای مورد استفاده قرار می‌گیرد. این تابع تمام مقادیر منفی در ورودی را با صفر جایگزین می‌کند و به مدل اجازه یادگیری الگوهای پیچیده را می‌دهد.
- **Leaky ReLU:** Leaky ReLU نوعی از ReLU است که یک گرادیان کوچک و غیرصفر را برای مقادیر ورودی منفی مجاز می‌کند. این به جلوگیری از نوروں‌های مرده (dead neurons) و تسهیل آموزش کمک می‌کند.
- **Parametric ReLU (PReLU):** PReLU یک پارامتر قابل یادگیری را معرفی می‌کند که شیب قسمت منفی تابع را تعیین می‌کند. این به مدل اجازه می‌دهد تا شیب را در طول آموزش تطبیق دهد.
- **Exponential Linear Unit (ELU):** ELU انتقال در حدود صفر را با وارد کردن یک شیب غیرصفر برای مقادیر منفی صاف می‌کند. این به کاهش مسائل مربوط به نوروں‌های مرده و تسریع همگرایی کمک می‌کند.

توابع فعال‌سازی واکنشی به پایداری و کارایی آموزش مدل‌های یادگیری عمیق کمک می‌کنند و در نهایت به بهبود عملکرد منجر می‌شوند.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک مدل یادگیری عمیق با استفاده از تابع فعال‌سازی Leaky ReLU:

In [4]: *# ReLU و Leaky ReLU آموزش: استفاده از توابع فعال‌سازی واکنشی*

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

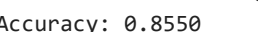
# 1 تولید داده مصنوعی برای دسته‌بندی
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2 ایجاد مدل یادگیری عمیق با توابع فعال‌سازی واکنشی
model = keras.Sequential([
    layers.Input(shape=(20,)),          # ورودی با ۲۰ ویژگی
    layers.Dense(64, activation='relu'), # ReLU لایه اول با
    layers.Dense(32),                  # لایه دوم بدون فعال‌سازی
    layers.LeakyReLU(negative_slope=0.1), # به صورت جداگانه Leaky ReLU اعمال
    layers.Dense(1, activation='sigmoid') # خروجی برای دسته‌بندی دودویی
])

# 3 کامپایل مدل
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 4 آموزش مدل
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_data=(X_test, y_test)
)

# 5 ارزیابی مدل
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

Epoch 1/10
25/25  2s 24ms/step - accuracy: 0.4985 - loss: 0.7070 - val_accuracy: 0.7550 - val_loss: 0.5891
Epoch 2/10
25/25  0s 5ms/step - accuracy: 0.8298 - loss: 0.5367 - val_accuracy: 0.8100 - val_loss: 0.4898
Epoch 3/10
25/25  0s 4ms/step - accuracy: 0.8618 - loss: 0.4346 - val_accuracy: 0.8150 - val_loss: 0.4201
Epoch 4/10
25/25  0s 6ms/step - accuracy: 0.8756 - loss: 0.3443 - val_accuracy: 0.8450 - val_loss: 0.3860
Epoch 5/10
25/25  0s 4ms/step - accuracy: 0.8838 - loss: 0.3267 - val_accuracy: 0.8450 - val_loss: 0.3713
Epoch 6/10
25/25  0s 4ms/step - accuracy: 0.8959 - loss: 0.2979 - val_accuracy: 0.8450 - val_loss: 0.3674
Epoch 7/10
25/25  0s 4ms/step - accuracy: 0.9058 - loss: 0.2492 - val_accuracy: 0.8400 - val_loss: 0.3706
Epoch 8/10
25/25  0s 6ms/step - accuracy: 0.9052 - loss: 0.2600 - val_accuracy: 0.8500 - val_loss: 0.3659
Epoch 9/10
25/25  0s 4ms/step - accuracy: 0.9013 - loss: 0.2555 - val_accuracy: 0.8350 - val_loss: 0.3634
Epoch 10/10
25/25  0s 6ms/step - accuracy: 0.9138 - loss: 0.2369 - val_accuracy: 0.8550 - val_loss: 0.3669
7/7  0s 4ms/step - accuracy: 0.8414 - loss: 0.3824
Test Accuracy: 0.8550

⚡ توضیحات مثال کد پایتون

در این مثال، یک شبکه عصبی ساده با استفاده از کتابخانه **TensorFlow/Keras** برای حل یک مسئله‌ی طبقه‌بندی دودویی طراحی شده است. تمرکز اصلی روی استفاده از توابع فعال‌سازی واکنشی مانند ReLU و Leaky ReLU است:

- ابتدا داده‌های مصنوعی با ۲۰ ویژگی و ۲ کلاس تولید شده‌اند.
- مدل شامل یک لایه با تابع **ReLU** و یک لایه با **Leaky ReLU** است که به‌صورت جداگانه اعمال شده‌اند.
- کامپایل مدل با بهینه‌ساز Adam و تابع زیان `binary_crossentropy` انجام شده است.
- مدل روی داده‌های آموزش آموزش دیده و روی داده‌های تست ارزیابی شده است.
- دقت نهایی مدل نشان‌دهنده عملکرد ترکیب توابع فعال‌سازی در یادگیری الگوهای پیچیده است.

استفاده از توابع فعال‌سازی واکنشی در معماری‌های عمیق باعث پایداری بیشتر در آموزش و جلوگیری از مشکلاتی مانند "نورون‌های مرده" می‌شود. این مثال پایه‌ای مناسب برای درک تفاوت بین ReLU و Leaky ReLU در عمل است.

10.2.2. Adaptive learning rate

🚀 نرخ یادگیری تطبیقی (Adaptive Learning Rate)

نرخ یادگیری (learning rate) یکی از هایپرپارامترهای کلیدی در آموزش مدل‌های یادگیری عمیق است که اندازه گام در فرآیند بهینه‌سازی را تعیین می‌کند. نرخ یادگیری تطبیقی در طول آموزش، خود را بر اساس پیشرفت مدل تنظیم می‌کند تا چالش‌هایی مانند همگرایی آهسته یا نوسانات تابع زیان (loss function) را کاهش دهد. برخی از الگوریتم‌های محبوب نرخ یادگیری تطبیقی عبارتند از:

- **Adagrad**: نرخ یادگیری هر پارامتر را بر اساس گرادیان‌های تاریخی آن تنظیم می‌کند. این الگوریتم با جمع‌آوری گرادیان‌های مجذور شده در طول زمان، نرخ‌های یادگیری مستقل برای هر پارامتر ارائه می‌دهد.
- **RMSprop**: نسخه اصلاح‌شده‌ای از Adagrad است که از میانگین متحرک گرادیان‌های مجذور شده استفاده می‌کند تا نرخ‌های یادگیری را به‌صورت روان‌تری تنظیم کند.

- **Adam**: ترکیبی از مومنتوم و RMSprop است که میانگین‌های متحرک گرادیان‌ها و مجذور آن‌ها را نگه می‌دارد و نرخ یادگیری را بر اساس آن‌ها تنظیم می‌کند. Adam یکی از پرکاربردترین بهینه‌سازها در یادگیری عمیق است.

این الگوریتم‌ها به مدل‌ها کمک می‌کنند تا سریع‌تر و پایدارتر همگرا شوند، به‌ویژه در مواجهه با داده‌های پراکنده یا نویزی.

مثال عملی در پایتون با استفاده از Adam برای یک مدل یادگیری عمیق:

In [6]: Adam آموزش: ساخت مدل یادگیری عمیق با بهینه‌ساز #

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

# تولید داده مصنوعی برای دسته‌بندی دودویی 1
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# تعریف مدل شبکه عصبی با دو لایه پنهان و تابع فعال‌سازی 2
model = keras.Sequential([
    layers.Dense(64, input_dim=20, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid') # خروجی برای دسته‌بندی دودویی
])

# تابع زیان باینری Adam کامپایل مدل با بهینه‌ساز 3
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست 4
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))

# ارزیابی مدل روی داده‌های تست 5
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
Epoch 1/10
25/25 ————— 3s 35ms/step - accuracy: 0.5508 - loss: 0.6787 - val_accuracy: 0.7700 - val_loss: 0.5587
Epoch 2/10
25/25 ————— 1s 8ms/step - accuracy: 0.8278 - loss: 0.4849 - val_accuracy: 0.8150 - val_loss: 0.4470
Epoch 3/10
25/25 ————— 0s 8ms/step - accuracy: 0.8609 - loss: 0.3760 - val_accuracy: 0.8250 - val_loss: 0.3954
Epoch 4/10
25/25 ————— 0s 7ms/step - accuracy: 0.8581 - loss: 0.3319 - val_accuracy: 0.8350 - val_loss: 0.3824
Epoch 5/10
25/25 ————— 0s 7ms/step - accuracy: 0.8844 - loss: 0.2858 - val_accuracy: 0.8450 - val_loss: 0.3744
Epoch 6/10
25/25 ————— 0s 6ms/step - accuracy: 0.8936 - loss: 0.2671 - val_accuracy: 0.8400 - val_loss: 0.3713
Epoch 7/10
25/25 ————— 0s 8ms/step - accuracy: 0.8815 - loss: 0.2655 - val_accuracy: 0.8450 - val_loss: 0.3723
Epoch 8/10
25/25 ————— 0s 7ms/step - accuracy: 0.8931 - loss: 0.2582 - val_accuracy: 0.8400 - val_loss: 0.3773
Epoch 9/10
25/25 ————— 0s 5ms/step - accuracy: 0.8948 - loss: 0.2684 - val_accuracy: 0.8450 - val_loss: 0.3775
Epoch 10/10
25/25 ————— 0s 4ms/step - accuracy: 0.9221 - loss: 0.2271 - val_accuracy: 0.8450 - val_loss: 0.3770
7/7 ————— 0s 5ms/step - accuracy: 0.8336 - loss: 0.4086
Test Accuracy: 0.8450
```

توضیحات مثال کد پایتون

در این مثال، یک مدل یادگیری عمیق ساده با استفاده از کتابخانه **TensorFlow/Keras** برای حل یک مسئله‌ی طبقه‌بندی دودویی طراحی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ۲۰ ویژگی و ۲ کلاس تولید شده‌اند.
- داده‌ها به دو بخش آموزش (۸۰٪) و آزمون (۲۰٪) تقسیم شده‌اند.
- مدل شامل دو لایه پنهان با توابع فعال‌سازی **ReLU** و یک لایه خروجی با **sigmoid** برای طبقه‌بندی دودویی است.
- کامپایل مدل با بهینه‌ساز **Adam** و تابع زیان **binary_crossentropy** انجام شده است.
- مدل روی داده‌های آموزش آموزش دیده و روی داده‌های تست ارزیابی شده است.

این ساختار پایه‌ای برای مدل‌های طبقه‌بندی دودویی در یادگیری عمیق است و می‌تواند به‌راحتی برای مسائل پیچیده‌تر توسعه یابد. استفاده از **Adam** باعث تنظیم خودکار نرخ یادگیری و همگرایی سریع‌تر مدل می‌شود.

10.2.3. Dropout

درآپاوت (Dropout)

درآپاوت (Dropout) یک تکنیک تنظیم‌کنندگی (regularization technique) است که در یادگیری عمیق برای جلوگیری از بیش‌برازش (overfitting) و بهبود تعمیم‌پذیری (generalization) مدل‌ها استفاده می‌شود. این روش شامل "غیرفعال کردن" تصادفی (تنظیم به صفر) کسری از نورون‌ها در طول آموزش است. این امر به جلوگیری از هم‌سازگاری (co-adaptation) نورون‌ها کمک کرده و شبکه را به یادگیری ویژگی‌های قوی‌تر تشویق می‌کند.

نکات کلیدی درباره درآپاوت:

- **غیرفعال‌سازی تصادفی نورون:** در هر تکرار آموزش، کسری تصادفی از نورون‌ها غیرفعال می‌شوند (خروجی آن‌ها صفر می‌شود)، هم در لایه‌های ورودی و هم در لایه‌های پنهان (hidden layers). این امر از وابستگی به نورون‌های خاص جلوگیری کرده و یک نمایش توزیع‌شده‌تر را ترویج می‌دهد.
- **اثر گروهی (Ensemble Effect):** درآپاوت را می‌توان به عنوان آموزش یک گروه (ensemble) از مدل‌ها در نظر گرفت، زیرا زیرمجموعه‌های مختلفی از نورون‌ها در هر تکرار حذف می‌شوند. این اثر گروهی به بهبود تعمیم‌پذیری کمک می‌کند.
- **تنظیم‌کنندگی (Regularization):** درآپاوت به عنوان شکلی از تنظیم‌کنندگی عمل می‌کند، با وارد کردن نویز و دلسرد کردن شبکه از به خاطر سپردن داده‌های آموزشی، خطر بیش‌برازش را کاهش می‌دهد.
- **قابلیت کاربرد (Applicability):** درآپاوت معمولاً در لایه‌های کاملاً متصل (fully connected layers) استفاده می‌شود، اما بسته به معماری شبکه می‌تواند در لایه‌های دیگر نیز به کار رود.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک مدل یادگیری عمیق با درآپاوت ارائه دهیم:

In [8]: در شبکه‌های عصبی Dropout آموزش: استفاده از لایه

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

# 1 تولید داده مصنوعی برای دسته‌بندی دودویی
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2 برای کاهش بیش‌برازش Dropout تعریف مدل شبکه عصبی با لایه‌های
model = keras.Sequential([
    layers.Dense(64, input_dim=20, activation='relu'),
    layers.Dropout(0.5), # حذف تصادفی ۵۰٪ نورون‌ها در آموزش
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid') # خروجی برای دسته‌بندی دودویی
])

# 3 و تابع زیان باینری Adam کامپایل مدل با بهینه‌ساز
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 4 آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))

# 5 ارزیابی مدل روی داده‌های تست
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

Epoch 1/10
25/25 ————— 3s 26ms/step - accuracy: 0.4982 - loss: 0.8058 - val_accuracy: 0.6550 - val_loss: 0.6187
Epoch 2/10
25/25 ————— 0s 5ms/step - accuracy: 0.5250 - loss: 0.7011 - val_accuracy: 0.7650 - val_loss: 0.5779
Epoch 3/10
25/25 ————— 0s 7ms/step - accuracy: 0.6428 - loss: 0.6205 - val_accuracy: 0.8100 - val_loss: 0.5384
Epoch 4/10
25/25 ————— 0s 7ms/step - accuracy: 0.7219 - loss: 0.5508 - val_accuracy: 0.8150 - val_loss: 0.5005
Epoch 5/10
25/25 ————— 0s 7ms/step - accuracy: 0.7065 - loss: 0.5386 - val_accuracy: 0.8150 - val_loss: 0.4695
Epoch 6/10
25/25 ————— 0s 6ms/step - accuracy: 0.7411 - loss: 0.5261 - val_accuracy: 0.8250 - val_loss: 0.4456
Epoch 7/10
25/25 ————— 0s 8ms/step - accuracy: 0.8232 - loss: 0.4379 - val_accuracy: 0.8350 - val_loss: 0.4225
Epoch 8/10
25/25 ————— 0s 7ms/step - accuracy: 0.8130 - loss: 0.4423 - val_accuracy: 0.8300 - val_loss: 0.4028
Epoch 9/10
25/25 ————— 0s 8ms/step - accuracy: 0.8025 - loss: 0.4433 - val_accuracy: 0.8350 - val_loss: 0.3903
Epoch 10/10
25/25 ————— 0s 5ms/step - accuracy: 0.8213 - loss: 0.4322 - val_accuracy: 0.8350 - val_loss: 0.3836
7/7 ————— 0s 4ms/step - accuracy: 0.8118 - loss: 0.3941
Test Accuracy: 0.8350

🗨 توضیحات مثال کد پایتون

در این مثال، یک مدل یادگیری عمیق با استفاده از لایه‌های **Dropout** طراحی شده است تا از بیش‌برازش (Overfitting) جلوگیری شود. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ۲۰ ویژگی و ۲ کلاس تولید شده‌اند.
- مدل شامل دو لایه پنهان با توابع فعال‌سازی **ReLU** و دو لایه **Dropout** با نرخ ۰.۵ است.
- Dropout در مرحله آموزش باعث حذف تصادفی نورون‌ها شده و مدل را به تعمیم بهتر وادار می‌کند.
- کامپایل مدل با بهینه‌ساز **Adam** و تابع زیان `binary_crossentropy` انجام شده است.
- مدل روی داده‌های آموزش آموزش دیده و روی داده‌های تست ارزیابی شده است.

استفاده از Dropout یکی از روش‌های مؤثر برای افزایش پایداری مدل و جلوگیری از یادگیری بیش‌ازحد جزئیات داده‌های آموزش است. این تکنیک در معماری‌های عمیق بسیار رایج و کاربردی است.

10.2.4. Pretraining

🎓 پیش‌آموزش (Pretraining)

پیش‌آموزش (Pretraining) تکنیکی در یادگیری عمیق است که در آن یک مدل در ابتدا بر روی یک وظیفه یا مجموعه داده مرتبط آموزش داده می‌شود، قبل از اینکه بر روی وظیفه هدف تنظیم دقیق (fine-tuned) گردد. این فرآیند به مدل اجازه می‌دهد تا ویژگی‌های عمومی (generic features) را از مجموعه داده اولیه یاد بگیرد، که می‌تواند زمانی که مجموعه داده هدف محدود است یا فاقد نمونه‌های دارای برچسب کافی است، مفید باشد.

نکات کلیدی درباره پیش‌آموزش:

- **آموزش اولیه بر روی یک وظیفه مرتبط:** مدل بر روی یک وظیفه یا مجموعه داده که با وظیفه هدف مرتبط است، پیش‌آموزش داده می‌شود. این می‌تواند یک مجموعه داده بزرگ‌تر با ویژگی‌های مشابه یا یک وظیفه مرتبط باشد که نمایش‌های زیربنایی مشترکی را به اشتراک می‌گذارد.
- **یادگیری ویژگی:** پیش‌آموزش مدل را قادر می‌سازد تا ویژگی‌ها و نمایش‌های عمومی را یاد بگیرد که به طور بالقوه قابل انتقال به وظیفه هدف هستند. لایه‌های اولیه شبکه الگوهای کلی را ثبت می‌کنند، در حالی که لایه‌های بعدی با وظیفه

خاص تطبیق می‌یابند.

- **تنظیم دقیق:** پس از پیش‌آموزش، مدل با استفاده از داده‌های دارای برچسب محدود موجود بر روی وظیفه هدف تنظیم دقیق می‌شود. این فرآیند به مدل کمک می‌کند تا با بهره‌گیری از دانشی که در طول پیش‌آموزش به دست آورده، برای وظیفه خاص تخصصی شود.
- **یادگیری انتقالی:** پیش‌آموزش شکلی از یادگیری انتقالی است، که در آن دانش به دست آمده از یک وظیفه به وظیفه دیگری منتقل می‌شود. این به ویژه در سناریوهایی مفید است که داده‌های دارای برچسب برای وظیفه هدف کمیاب هستند.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای پیش‌آموزش و تنظیم دقیق با استفاده از یادگیری انتقالی با یک مدل از پیش‌آموزش دیده ارائه دهیم:

In [10]: *# VGG16 با (Pretraining + Fine-tuning) آموزش: پیش‌آموزش و تنظیم دقیق*

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.applications import VGG16
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.datasets import cifar10

# 1 و انتخاب دو کلاس برای دسته‌بندی دودویی (CIFAR-10) بارگذاری داده واقعی
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
binary_classes = [0, 1]
train_mask = np.isin(y_train, binary_classes).flatten()
test_mask = np.isin(y_test, binary_classes).flatten()
X_train, y_train = X_train[train_mask], y_train[train_mask]
X_test, y_test = X_test[test_mask], y_test[test_mask]
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
y_train = y_train.flatten()
y_test = y_test.flatten()

# 2 پیش‌آموزش دیده بدون لایه‌های بالایی VGG16 بارگذاری مدل
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(32, 32, 3))
for layer in base_model.layers:
    layer.trainable = False # فریز کردن لایه‌ها برای جلوگیری از تغییر وزن‌ها

# 3 اضافه کردن لایه‌های جدید برای دسته‌بندی دودویی
model = keras.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

# 4 کامپایل مدل
model.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# 5 آموزش مدل
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_data=(X_test, y_test)
)

# 6 ارزیابی مدل
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```


Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 27s 0us/step
Epoch 1/10
313/313 12s 23ms/step - accuracy: 0.6606 - loss: 0.5987 - val_accuracy: 0.8795 - val_loss: 0.3236
Epoch 2/10
313/313 13s 10ms/step - accuracy: 0.8693 - loss: 0.3262 - val_accuracy: 0.8915 - val_loss: 0.2723
Epoch 3/10
313/313 3s 9ms/step - accuracy: 0.8880 - loss: 0.2835 - val_accuracy: 0.9005 - val_loss: 0.2501
Epoch 4/10
313/313 5s 9ms/step - accuracy: 0.8904 - loss: 0.2634 - val_accuracy: 0.9050 - val_loss: 0.2394
Epoch 5/10
313/313 3s 10ms/step - accuracy: 0.8976 - loss: 0.2523 - val_accuracy: 0.9075 - val_loss: 0.2290
Epoch 6/10
313/313 5s 9ms/step - accuracy: 0.9058 - loss: 0.2402 - val_accuracy: 0.9120 - val_loss: 0.2221
Epoch 7/10
313/313 3s 9ms/step - accuracy: 0.9082 - loss: 0.2330 - val_accuracy: 0.9095 - val_loss: 0.2192
Epoch 8/10
313/313 6s 10ms/step - accuracy: 0.9066 - loss: 0.2265 - val_accuracy: 0.9150 - val_loss: 0.2119
Epoch 9/10
313/313 3s 9ms/step - accuracy: 0.9170 - loss: 0.2143 - val_accuracy: 0.9170 - val_loss: 0.2079
Epoch 10/10
313/313 3s 9ms/step - accuracy: 0.9187 - loss: 0.2075 - val_accuracy: 0.9145 - val_loss: 0.2045
63/63 1s 9ms/step - accuracy: 0.9147 - loss: 0.2053
Test Accuracy: 0.9145

توضیحات مثال کد پایتون

در این مثال، از مدل **VGG16** که قبلاً روی مجموعه داده ImageNet آموزش دیده، برای استخراج ویژگی‌های عمومی استفاده شده است. سپس لایه‌های جدیدی برای دسته‌بندی دودویی اضافه شده‌اند و فقط همین لایه‌ها آموزش داده شده‌اند:

- ابتدا داده‌های CIFAR-10 بارگذاری شده و فقط دو کلاس (۰ و ۱) برای دسته‌بندی انتخاب شده‌اند.
- مدل VGG16 بدون لایه‌های بالایی بارگذاری شده و وزن‌های آن فریز شده‌اند تا در آموزش تغییر نکنند.
- لایه‌های جدید شامل Flatten، Dense، Dropout و خروجی sigmoid برای دسته‌بندی دودویی اضافه شده‌اند.
- مدل فقط روی لایه‌های جدید آموزش دیده و روی داده‌های تست ارزیابی شده است.

این فرآیند نمونه‌ای از **یادگیری انتقالی** است که در آن از دانش قبلی مدل‌های بزرگ برای حل مسائل جدید استفاده می‌شود. این روش باعث صرفه‌جویی در زمان آموزش و بهبود عملکرد در داده‌های کم‌حجم می‌شود، پنی‌ک نکنید چون جلوتر transfer learning رو یاد می‌گیرید 😊

10.2.5. Cross-entropy

آنتروپی متقاطع (Cross-entropy)

آنتروپی متقاطع (Cross-entropy)، که اغلب به عنوان یک تابع زیان (loss function) استفاده می‌شود، معیاری برای اندازه‌گیری تفاوت بین دو توزیع احتمال (probability distributions) است. در زمینه یادگیری عمیق، معمولاً به عنوان تابع زیان برای وظایف طبقه‌بندی (classification tasks) استفاده می‌شود. زیان آنتروپی متقاطع (cross-entropy loss) میزان هم‌راستایی توزیع احتمال پیش‌بینی شده با توزیع واقعی برچسب‌های هدف را کمی‌سازی می‌کند.

نکات کلیدی درباره آنتروپی متقاطع:

- **آنتروپی متقاطع دودویی:** برای وظایف طبقه‌بندی دودویی (binary classification)، از آنتروپی متقاطع دودویی استفاده می‌شود. این معیار، عدم شباهت بین توزیع احتمال پیش‌بینی شده و برچسب‌های دودویی واقعی را اندازه‌گیری می‌کند.

- **آنتروپی متقاطع طبقه‌ای:** برای وظایف طبقه‌بندی چندکلاسه (multi-class classification)، از آنتروپی متقاطع طبقه‌ای استفاده می‌شود. این معیار، آنتروپی متقاطع دودویی را برای مدیریت چندین کلاس گسترش می‌دهد.
- **تفسیر نظریه اطلاعات:** آنتروپی متقاطع از نظریه اطلاعات (information theory) مشتق شده و میانگین تعداد بیت‌های مورد نیاز برای نمایش یک رویداد از یک توزیع را، هنگام استفاده از کدگذاری بهینه مبتنی بر توزیع دیگر، اندازه‌گیری می‌کند.

- **هدف آموزش:** حداقل کردن آنتروپی متقاطع در طول آموزش با هدف نزدیک‌تر کردن توزیع احتمال پیش‌بینی شده به توزیع واقعی عمل می‌کند و توانایی مدل برای طبقه‌بندی دقیق را بهبود می‌بخشد.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک مدل یادگیری عمیق با آنتروپی متقاطع به عنوان تابع زیان:

In [12]: `binary_crossentropy آموزش: ساخت مدل یادگیری عمیق برای دسته‌بندی دودویی با تابع زیان`

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

# تولید داده مصنوعی برای دسته‌بندی دودویی 1
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# تعریف مدل شبکه عصبی با دو لایه پنهان و خروجی 2
model = keras.Sequential([
    layers.Dense(64, input_dim=20, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid') # خروجی برای دسته‌بندی دودویی
])

# بهینه‌ساز Adam و بهینه‌ساز binary_crossentropy کامپایل مدل با تابع زیان 3
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست 4
model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test))

# ارزیابی مدل روی داده‌های تست 5
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
Epoch 1/10
25/25 ————— 2s 35ms/step - accuracy: 0.6245 - loss: 0.6359 - val_accuracy: 0.7650 - val_loss: 0.5270
Epoch 2/10
25/25 ————— 1s 5ms/step - accuracy: 0.8293 - loss: 0.4576 - val_accuracy: 0.8100 - val_loss: 0.4496
Epoch 3/10
25/25 ————— 0s 4ms/step - accuracy: 0.8515 - loss: 0.3846 - val_accuracy: 0.8250 - val_loss: 0.4045
Epoch 4/10
25/25 ————— 0s 5ms/step - accuracy: 0.8686 - loss: 0.3409 - val_accuracy: 0.8400 - val_loss: 0.3785
Epoch 5/10
25/25 ————— 0s 5ms/step - accuracy: 0.8596 - loss: 0.3265 - val_accuracy: 0.8450 - val_loss: 0.3683
Epoch 6/10
25/25 ————— 0s 4ms/step - accuracy: 0.8599 - loss: 0.3318 - val_accuracy: 0.8350 - val_loss: 0.3696
Epoch 7/10
25/25 ————— 0s 4ms/step - accuracy: 0.9049 - loss: 0.2565 - val_accuracy: 0.8350 - val_loss: 0.3601
Epoch 8/10
25/25 ————— 0s 5ms/step - accuracy: 0.8976 - loss: 0.2725 - val_accuracy: 0.8350 - val_loss: 0.3600
Epoch 9/10
25/25 ————— 0s 6ms/step - accuracy: 0.9156 - loss: 0.2504 - val_accuracy: 0.8650 - val_loss: 0.3548
Epoch 10/10
25/25 ————— 0s 7ms/step - accuracy: 0.8950 - loss: 0.2593 - val_accuracy: 0.8450 - val_loss: 0.3549
7/7 ————— 0s 7ms/step - accuracy: 0.8118 - loss: 0.3802
Test Accuracy: 0.8450
```

توضیحات مثال کد پایتون

در این مثال، یک مدل یادگیری عمیق ساده با استفاده از کتابخانه **TensorFlow/Keras** برای حل یک مسئله‌ی طبقه‌بندی دودویی طراحی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ۲۰ ویژگی و ۲ کلاس تولید شده‌اند.
- مدل شامل دو لایه پنهان با توابع فعال‌سازی **ReLU** و یک لایه خروجی با **sigmoid** برای طبقه‌بندی دودویی است.
- کامپایل مدل با تابع زیان `binary_crossentropy` و بهینه‌ساز **Adam** انجام شده است.
- مدل روی داده‌های آموزش آموزش دیده و روی داده‌های تست ارزیابی شده است.

این ساختار پایه‌ای برای مدل‌های طبقه‌بندی دودویی در یادگیری عمیق است و می‌تواند به راحتی برای مسائل پیچیده‌تر توسعه یابد. استفاده از **binary_crossentropy** برای خروجی‌های باینری ضروری است و باعث همگرایی بهتر مدل می‌شود.

10.2.6. Autoencoder: unsupervised deep learning

خودرمزگذار (Autoencoder) ⚙️

خودرمزگذار (Autoencoder) نوعی مدل یادگیری بدون نظارت (unsupervised learning) در یادگیری عمیق است که برای یادگیری نمایش‌های کارآمد از داده‌ها طراحی شده است. این مدل شامل یک رمزگذار (encoder) و یک رمزگشا (decoder) است که با هم برای بازسازی داده ورودی کار می‌کنند. رمزگذار، ورودی را به یک نمایش فضای پنهان (latent space representation) فشرده می‌کند و رمزگشا، ورودی را از این نمایش بازسازی می‌نماید.

نکات کلیدی درباره خودرمزگذار:

- **رمزگذار (Encoder):** شبکه رمزگذار، داده ورودی را به یک نمایش با ابعاد پایین‌تر، که به عنوان فضای پنهان یا رمزگذاری شناخته می‌شود، نگاشت می‌کند. این فرآیند ویژگی‌های ضروری ورودی را ثبت می‌نماید.
- **رمزگشا (Decoder):** شبکه رمزگشا داده ورودی را از رمزگذاری بازسازی می‌کند. هدف، تولید خروجی است که تا حد امکان به ورودی اصلی نزدیک باشد.
- **تابع زیان (Loss Function):** تابع زیان مورد استفاده در طول آموزش، تفاوت بین ورودی و خروجی بازسازی شده را اندازه‌گیری می‌کند. گزینه‌های رایج شامل میانگین خطای مربعات (mean squared error) برای داده‌های با مقادیر حقیقی یا آنتروپی متقاطع دودویی (binary cross-entropy) برای داده‌های دودویی هستند.
- **کاربردها (Applications):** خودرمزگذارها کاربردهای متنوعی از جمله حذف نویز از داده‌ها (data denoising)، کاهش ابعاد (dimensionality reduction) و تشخیص ناهنجاری (anomaly detection) دارند. آن‌ها به ویژه زمانی مفید هستند که داده‌های دارای برچسب کمیاب یا در دسترس نباشند.
- **انواع (Variations):** انواع خودرمزگذارها شامل خودرمزگذارهای پراکنده (sparse autoencoders)، خودرمزگذارهای حذف نویز (denoising autoencoders) و خودرمزگذارهای واریشنال (variational autoencoders) هستند که هر یک برای اهداف خاصی طراحی شده‌اند.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک خودرمزگذار:

In [14]: ساده برای بازسازی داده‌ها (Autoencoder) آموزش: ساخت خودرمزگذار #

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

# 1 تولید داده مصنوعی برای دسته‌بندی دودویی
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2 تعریف معماری خودرمزگذار با لایه فشرده‌سازی و بازسازی
input_dim = X_train.shape[1]
encoding_dim = 10 # ابعاد فشرده‌شده
autoencoder = keras.Sequential([
    layers.Dense(encoding_dim, input_dim=input_dim, activation='relu'), # لایه فشرده‌سازی
    layers.Dense(input_dim, activation='sigmoid') # لایه بازسازی
])

# 3 Adam و بهینه‌ساز binary_crossentropy کامپایل مدل با تابع زیان
```

```
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')

# آموزش مدل روی داده‌های ورودی و بازسازی همان داده‌ها
autoencoder.fit(X_train, X_train, epochs=10, batch_size=32, validation_data=(X_test, X_test))

# ارزیابی عملکرد مدل با میانگین خطای مربعی روی داده‌های تست
encoded_data = autoencoder.predict(X_test)
mse = np.mean(np.square(X_test - encoded_data))
print(f"Mean Squared Error on Test Set: {mse:.6f}")
```

Epoch 1/10
25/25 ————— 2s 33ms/step - loss: 0.7083 - val_loss: 0.6440
Epoch 2/10
25/25 ————— 1s 4ms/step - loss: 0.6258 - val_loss: 0.5688
Epoch 3/10
25/25 ————— 0s 4ms/step - loss: 0.5534 - val_loss: 0.4968
Epoch 4/10
25/25 ————— 0s 4ms/step - loss: 0.4766 - val_loss: 0.4250
Epoch 5/10
25/25 ————— 0s 4ms/step - loss: 0.3976 - val_loss: 0.3513
Epoch 6/10
25/25 ————— 0s 4ms/step - loss: 0.3285 - val_loss: 0.2743
Epoch 7/10
25/25 ————— 0s 6ms/step - loss: 0.2381 - val_loss: 0.1939
Epoch 8/10
25/25 ————— 0s 5ms/step - loss: 0.1656 - val_loss: 0.1102
Epoch 9/10
25/25 ————— 0s 4ms/step - loss: 0.0590 - val_loss: 0.0217
Epoch 10/10
25/25 ————— 0s 4ms/step - loss: -0.0185 - val_loss: -0.0704
7/7 ————— 0s 25ms/step
Mean Squared Error on Test Set: 1.013979

🔒 توضیحات مثال کد پایتون

در این مثال، یک مدل **خودرمزگذار (Autoencoder)** ساده برای بازسازی داده‌های ورودی طراحی شده است. هدف این مدل یادگیری نمایش فشرده‌ای از داده‌ها و بازسازی آن‌ها با کمترین خطا است:

- ابتدا داده‌های مصنوعی با ۲۰ ویژگی تولید شده‌اند و به دو بخش آموزش و تست تقسیم شده‌اند.
- مدل شامل یک لایه فشرده‌سازی با ۱۰ نورون و یک لایه بازسازی با خروجی برابر با ورودی است.
- مدل با تابع زیان `binary_crossentropy` آموزش دیده و هدف آن بازسازی دقیق داده‌های ورودی است.
- در پایان، عملکرد مدل با استفاده از **خطای میانگین مربعات (MSE)** روی داده‌های تست ارزیابی شده است.

خودرمزگذارها در کاربردهایی مانند کاهش ابعاد، تشخیص ناهنجاری، و استخراج ویژگی‌های پنهان بسیار مؤثر هستند. این مثال پایه‌ای مناسب برای درک معماری و عملکرد Autoencoder است.

Section 10.3

10.3.1. Introducing convolution operation

📷 عملیات کانولوشن (The Convolution Operation)

عملیات کانولوشن یک بلوک ساختمانی بنیادی در شبکه‌های عصبی کانولوشنال (Convolutional Neural Networks - CNNs) است. CNN ها به ویژه برای وظایف مرتبط با تحلیل تصویر مؤثر هستند، اما کاربردهایی در حوزه‌های مختلف پیدا کرده‌اند. عملیات کانولوشن شامل اعمال یک فیلتر (که به آن هسته یا kernel نیز گفته می‌شود) بر روی داده ورودی است، که به شبکه امکان می‌دهد الگوهای محلی و سلسله‌مراتب فضایی را ثبت کند.

نکات کلیدی درباره عملیات کانولوشن:

- **استخراج ویژگی محلی:** عملیات کانولوشن بر روی نواحی محلی ورودی تمرکز می‌کند و به شبکه اجازه می‌دهد ویژگی‌های خاصی را ثبت نماید. این در مقابل لایه‌های کاملاً متصل (fully connected layers) است که کل ورودی را به طور همزمان در نظر می‌گیرند.
- **اشتراک وزن‌ها:** یک فیلتر یکسان در مکان‌های فضایی مختلف اعمال می‌شود که اشتراک وزن (weight sharing) را ترویج می‌دهد. این امر تعداد پارامترها را کاهش می‌دهد و به شبکه اجازه می‌دهد سلسله‌مراتب فضایی را به طور کارآمدی یاد بگیرد.
- **نقشه‌های ویژگی:** خروجی عملیات کانولوشن، نقشه ویژگی (feature map) نامیده می‌شود. هر عنصر در نقشه ویژگی، نشان‌دهنده وجود یک ویژگی خاص در ورودی است.
- **پولینگ:** لایه‌های پولینگ (Pooling layers) اغلب پس از کانولوشن برای نمونه‌برداری به پایین (downsample) از ابعاد فضایی و کاهش پیچیدگی محاسباتی استفاده می‌شوند.
- **کاربردها:** CNN‌ها به طور گسترده برای طبقه‌بندی تصویر، تشخیص شیء (object detection)، بخش‌بندی (segmentation) و وظایف مختلف بینایی کامپیوتری استفاده می‌شوند. آن‌ها همچنین در وظایف پردازش زبان طبیعی و تشخیص گفتار نیز به کار رفته‌اند.

اکنون، بیا یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک شبکه عصبی کانولوشنال با استفاده از Keras و TensorFlow ارائه دهیم:

In [15]: *برای دسته‌بندی تصاویر ارقام (CNN) آموزش: ساخت شبکه عصبی کانولوشنی*

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

# 1 برای دسته‌بندی تصاویر ارقام digits بارگذاری مجموعه داده
digits = load_digits()
X, y = digits.images, digits.target

# 2 تقسیم داده‌ها به آموزش و تست
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3 تغییر شکل تصاویر برای داشتن کانال تک‌بعدی (سیاه و سفید)
X_train = X_train.reshape(-1, 8, 8, 1)
X_test = X_test.reshape(-1, 8, 8, 1)

# 4 Pooling ساده با لایه‌های کانولوشن و CNN تعریف مدل
model = keras.Sequential([
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(8, 8, 1)),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax') # خروجی برای ۱۰ کلاس
])

# 5 Adam و بهینه‌ساز categorical کامپایل مدل با تابع زیان
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# 6 آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست
model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test))

# 7 ارزیابی مدل روی داده‌های تست
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

Epoch 1/10

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)


```
45/45 ————— 4s 43ms/step - accuracy: 0.4655 - loss: 1.6963 - val_accuracy: 0.9139 - val_loss: 0.2984
Epoch 2/10
45/45 ————— 0s 4ms/step - accuracy: 0.9361 - loss: 0.2569 - val_accuracy: 0.9556 - val_loss: 0.1425
Epoch 3/10
45/45 ————— 0s 4ms/step - accuracy: 0.9694 - loss: 0.1291 - val_accuracy: 0.9861 - val_loss: 0.0847
Epoch 4/10
45/45 ————— 0s 4ms/step - accuracy: 0.9850 - loss: 0.0752 - val_accuracy: 0.9806 - val_loss: 0.0761
Epoch 5/10
45/45 ————— 0s 4ms/step - accuracy: 0.9857 - loss: 0.0584 - val_accuracy: 0.9861 - val_loss: 0.0572
Epoch 6/10
45/45 ————— 0s 4ms/step - accuracy: 0.9965 - loss: 0.0322 - val_accuracy: 0.9833 - val_loss: 0.0772
Epoch 7/10
45/45 ————— 0s 4ms/step - accuracy: 0.9946 - loss: 0.0342 - val_accuracy: 0.9833 - val_loss: 0.0477
Epoch 8/10
45/45 ————— 0s 4ms/step - accuracy: 0.9972 - loss: 0.0229 - val_accuracy: 0.9806 - val_loss: 0.0478
Epoch 9/10
45/45 ————— 0s 4ms/step - accuracy: 0.9987 - loss: 0.0178 - val_accuracy: 0.9861 - val_loss: 0.0399
Epoch 10/10
45/45 ————— 0s 4ms/step - accuracy: 0.9991 - loss: 0.0118 - val_accuracy: 0.9833 - val_loss: 0.0375
12/12 ————— 0s 5ms/step - accuracy: 0.9887 - loss: 0.0348
Test Accuracy: 0.9833333492279053
```

توضیحات مثال کد پایتون

در این مثال، یک شبکه عصبی کانولوشنی (CNN) ساده برای دسته‌بندی تصاویر ارقام از مجموعه داده **digits** طراحی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا تصاویر 8×8 از مجموعه داده digits بارگذاری شده‌اند و به دو بخش آموزش و تست تقسیم شده‌اند.
- تصاویر به شکل مناسب برای ورودی شبکه کانولوشنی (با کانال تک‌بعدی) تبدیل شده‌اند.
- مدل شامل یک لایه کانولوشن، یک لایه Pooling، و دو لایه Dense است که در نهایت خروجی softmax برای ۱۰ کلاس تولید می‌کند.
- مدل با تابع زیان `sparse_categorical_crossentropy` و بهینه‌ساز **Adam** کامپایل شده و آموزش دیده است.
- در پایان، دقت مدل روی داده‌های تست محاسبه شده است.

این مثال پایه‌ای مناسب برای درک معماری‌های CNN در مسائل دسته‌بندی تصویری است و می‌تواند به راحتی برای داده‌های پیچیده‌تر مانند CIFAR یا MNIST توسعه یابد.

10.3.2. Multidimensional convolution

کانولوشن چندبعدی (Multidimensional Convolution)

کانولوشن چندبعدی، مفهوم کانولوشن را به ابعاد چندگانه گسترش می‌دهد و شبکه‌های عصبی کانولوشنال (Convolutional Neural Networks - CNNs) را قادر می‌سازد داده‌ها را با ساختارهای فضایی در بیش از یک بعد پردازش کنند. در حالی که کانولوشن 2D معمولاً برای داده‌های تصویری استفاده می‌شود، کانولوشن چندبعدی برای داده‌های حجمی (volumetric data) مانند تصاویر 3D یا توالی‌های دارای وابستگی‌های زمانی قابل استفاده است.

نکات کلیدی درباره کانولوشن چندبعدی:

- کانولوشن 3D:** علاوه بر ارتفاع و عرض، کانولوشن 3D بعد عمق (یا زمان در مورد توالی‌ها) را در نظر می‌گیرد. این شامل اعمال یک فیلتر 3D بر روی داده ورودی است که الگوهای فضازمانی (spatiotemporal) را ثبت می‌کند.
- کاربردها:** کانولوشن چندبعدی به ویژه برای وظایف مرتبط با داده‌های حجمی مانند تصویربرداری پزشکی (اسکن‌های 3D CT یا MRI)، تحلیل ویدئو و به طور کلی داده‌های فضازمانی مفید است.

- **عمق فیلتر:** فیلترهای مورد استفاده در کانولوشن چندبعدی دارای یک بعد عمق هستند که با داده ورودی مطابقت دارد. هر عنصر در بعد عمق فیلتر با یک برش از داده ورودی مطابقت دارد.
- **گام‌ها و پدینگ:** مشابه کانولوشن 2D، کانولوشن چندبعدی می‌تواند از گام‌ها (strides) و پدینگ (padding) برای کنترل ابعاد فضایی نقشه ویژگی خروجی استفاده کند.
- **مثال:** در حالی که کانولوشن 2D برای داده‌های تصویری رایج است، کانولوشن 3D هنگام در نظر گرفتن داده‌های حجمی اعمال می‌شود و این مفهوم برای ساختارهای داده پیچیده‌تر به ابعاد بالاتر گسترش می‌یابد.

در مثال پایین، داده‌های حجمی آماده‌سازی و نرمال‌سازی شدند، سپس با یک شبکه **3D CNN** ویژگی‌های فضازمانی استخراج و نمونه‌ها دسته‌بندی شدند.

In [33]: *# smaLLNORB شبه‌سه‌بعدی روی داده‌های (3D CNN) آموزش: ساخت شبکه عصبی کانولوشنی سه‌بعدی*

```
import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers, models

# 1 بارگذاری مجموعه داده smaLLNORB
(ds_train, ds_test), ds_info = tfds.load(
    'smallnorb',
    split=['train', 'test'],
    shuffle_files=True,
    as_supervised=False,
    with_info=True
)

NUM_CLASSES = ds_info.features['label_category'].num_classes
print("Number of classes:", NUM_CLASSES)

# 2 پیش‌پردازش تصاویر: نرمال‌سازی، تبدیل به استریو، تغییر اندازه، و افزودن بُعد کانال
def preprocess(sample):
    image = tf.cast(sample['image'], tf.float32) / 255.0
    image = tf.repeat(image, repeats=2, axis=-1) # تبدیل به تصویر استریو با ۲ کانال
    image = tf.image.resize(image, [32, 32]) # تغییر اندازه به ۳۲×۳۲
    image = tf.expand_dims(image, axis=-1) # 3D تبدیل به شکل (32,32,2,1) برای 3
    label = sample['label_category']
    return image, label

BATCH_SIZE = 32
ds_train = ds_train.map(preprocess).shuffle(1000).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
ds_test = ds_test.map(preprocess).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

# 3 با ورودی شبه‌سه‌بعدی 3D CNN تعریف مدل
model = models.Sequential([
    layers.Input(shape=(32, 32, 2, 1)), # height, width, depth, channels
    layers.Conv3D(32, kernel_size=(3, 3, 2), activation='relu', padding='valid'),
    layers.MaxPooling3D(pool_size=(2, 2, 1)), # pooling حفظ عمق با ۱ در عمق=۱
    layers.Conv3D(64, kernel_size=(3, 3, 1), activation='relu', padding='valid'),
    layers.MaxPooling3D(pool_size=(2, 2, 1)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(NUM_CLASSES, activation='softmax')
])

# 4 Adam و بهینه‌ساز categorical کامپایل مدل با تابع زیان
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# 5 آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست
EPOCHS = 10
history = model.fit(ds_train, validation_data=ds_test, epochs=EPOCHS)

# 6 ارزیابی مدل روی داده‌های تست
test_loss, test_acc = model.evaluate(ds_test)
print(f"Test Accuracy: {test_acc:.4f}")
```

Number of classes: 5
Model: "sequential_26"

Layer (type)	Output Shape	Param #
conv3d_17 (Conv3D)	(None, 30, 30, 1, 32)	608
max_pooling3d_17 (MaxPooling3D)	(None, 15, 15, 1, 32)	0
conv3d_18 (Conv3D)	(None, 13, 13, 1, 64)	18,496
max_pooling3d_18 (MaxPooling3D)	(None, 6, 6, 1, 64)	0
flatten_14 (Flatten)	(None, 2304)	0
dense_62 (Dense)	(None, 128)	295,040
dense_63 (Dense)	(None, 5)	645

Total params: 314,789 (1.20 MB)

Trainable params: 314,789 (1.20 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10
760/760 24s 28ms/step - accuracy: 0.6653 - loss: 0.8088 - val_accuracy: 0.8311 - val_loss: 0.4677
Epoch 2/10
760/760 10s 13ms/step - accuracy: 0.9606 - loss: 0.1131 - val_accuracy: 0.8313 - val_loss: 0.5935
Epoch 3/10
760/760 10s 13ms/step - accuracy: 0.9817 - loss: 0.0590 - val_accuracy: 0.8647 - val_loss: 0.5333
Epoch 4/10
760/760 8s 11ms/step - accuracy: 0.9905 - loss: 0.0291 - val_accuracy: 0.8938 - val_loss: 0.4683
Epoch 5/10
760/760 9s 11ms/step - accuracy: 0.9923 - loss: 0.0251 - val_accuracy: 0.8602 - val_loss: 0.7051
Epoch 6/10
760/760 9s 11ms/step - accuracy: 0.9952 - loss: 0.0143 - val_accuracy: 0.8577 - val_loss: 0.6527
Epoch 7/10
760/760 10s 13ms/step - accuracy: 0.9924 - loss: 0.0230 - val_accuracy: 0.8809 - val_loss: 0.6312
Epoch 8/10
760/760 10s 13ms/step - accuracy: 0.9958 - loss: 0.0123 - val_accuracy: 0.8823 - val_loss: 0.6520
Epoch 9/10
760/760 8s 10ms/step - accuracy: 0.9996 - loss: 0.0020 - val_accuracy: 0.8937 - val_loss: 0.6668
Epoch 10/10
760/760 9s 11ms/step - accuracy: 0.9951 - loss: 0.0151 - val_accuracy: 0.8489 - val_loss: 0.8840
760/760 3s 4ms/step - accuracy: 0.8489 - loss: 0.8630
Test Accuracy: 0.8489

توضیحات مثال کد پایتون

در این مثال، یک شبکه عصبی کانولوشنی سه‌بعدی (3D CNN) برای دسته‌بندی تصاویر استریو از مجموعه داده smallINORB طراحی شده است. این معماری به‌صورت شبه‌سه‌بعدی عمل می‌کند و از عمق تصویر به‌عنوان بُعد سوم استفاده می‌کند:

- ابتدا تصاویر نرمال‌سازی شده، به دو کانال (استریو) تبدیل شده و به اندازه ۳۲×۳۲ تغییر داده شده‌اند.
- تصاویر به شکل (32,32,2,1) تبدیل شده‌اند تا بتوان از لایه‌های Conv3D استفاده کرد.
- مدل شامل دو لایه کانولوشن سه‌بعدی، دو لایه Pooling، و دو لایه Dense است که در نهایت خروجی softmax برای دسته‌بندی چندکلاسه تولید می‌کند.
- کامپایل مدل با تابع زیان sparse_categorical_crossentropy و بهینه‌ساز Adam انجام شده است.
- مدل روی داده‌های آموزش آموزش دیده و روی داده‌های تست ارزیابی شده است.

استفاده از معماری‌های سه‌بعدی در مسائل بینایی ماشین با داده‌های استریو یا حجمی می‌تواند به استخراج ویژگی‌های فضایی دقیق‌تر کمک کند. این مثال پایه‌ای مناسب برای ورود به دنیای 3D CNN در داده‌های تصویری پیچیده‌تر است.

10.3.3. Convolutional layer

لایه کانولوشنال (Convolutional Layer)

لایه کانولوشنال (Convolutional layer) یک مؤلفه اصلی در شبکه‌های عصبی کانولوشنال (Convolutional Neural Networks - CNNs) است که مسئول یادگیری الگوهای محلی و نمایش‌های سلسله‌مراتبی در داده ورودی می‌باشد. این لایه با اعمال فیلترها بر روی ورودی، عملیات کانولوشن را انجام می‌دهد و شبکه را قادر می‌سازد سلسله‌مراتب فضایی و ویژگی‌ها را ثبت کند.

نکات کلیدی درباره لایه کانولوشنال:

- **فیلترها (هسته‌ها):** لایه کانولوشنال از فیلترها (که به آن‌ها هسته یا kernels نیز گفته می‌شود) برای اسکن داده ورودی استفاده می‌کند. این فیلترها ماتریس‌های کوچک و قابل یادگیری هستند که بر روی ورودی می‌لغزند و الگوهای محلی را ثبت می‌کنند.
- **میدان‌های گیرنده محلی:** هر فیلتر بر روی یک میدان گیرنده محلی از ورودی تمرکز می‌کند. با اشتراک وزن‌ها در سراسر میدان گیرنده، شبکه یاد می‌گیرد الگوهای مشابه را در مکان‌های فضایی مختلف تشخیص دهد.
- **گام‌ها و پدینگ:** گام‌ها (Strides) اندازه گام فیلتر را هنگام حرکت بر روی ورودی کنترل می‌کنند و بر ابعاد فضایی نقشه ویژگی خروجی تأثیر می‌گذارند. پدینگ (Padding) می‌تواند برای حفظ ابعاد فضایی استفاده شود.
- **تابع فعال‌سازی:** معمولاً یک تابع فعال‌سازی غیرخطی (مانند ReLU) پس از عملیات کانولوشن قرار می‌گیرد که ناخطی‌بودن (non-linearity) را به مدل وارد کرده و آن را قادر به یادگیری نمایش‌های پیچیده می‌سازد.
- **عمق:** عمق لایه کانولوشنال با تعداد فیلترهای اعمال شده مطابقت دارد. هر فیلتر یک نقشه ویژگی (feature map) تولید می‌کند و عمق لایه برابر با تعداد نقشه‌های ویژگی است.
- **پولینگ:** لایه‌های پولینگ (Pooling layers) اغلب پس از لایه‌های کانولوشنال برای نمونه‌برداری به پایین (downsample) از ابعاد فضایی نقشه‌های ویژگی و کاهش محاسبات استفاده می‌شوند.

اکنون، بیایید یک مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک لایه کانولوشنال در یک CNN ارائه دهیم:

In [34]: *# برای دسته‌بندی تصاویر ارقام (CNN) آموزش: ساخت شبکه عصبی کانولوشنی*

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from tensorflow import keras
from tensorflow.keras import layers

# 1 برای دسته‌بندی تصاویر ارقام digits بارگذاری مجموعه داده
digits = load_digits()
X, y = digits.images, digits.target

# 2 تقسیم داده‌ها به آموزش و تست
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3 تغییر شکل تصاویر برای داشتن کانال تک‌بعدی (سیاه و سفید)
X_train = X_train.reshape(-1, 8, 8, 1)
X_test = X_test.reshape(-1, 8, 8, 1)

# 4 Pooling ساده با یک لایه کانولوشن و CNN تعریف مدل
model = keras.Sequential([
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(8, 8, 1)),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax') # خروجی برای ۱۰ کلاس
])

# 5 Adam و بهینه‌ساز categorical کامپایل مدل با تابع زیان
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# 6 آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست
model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test))

# 7 ارزیابی مدل روی داده‌های تست
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

Epoch 1/10
45/45 3s 37ms/step - accuracy: 0.3861 - loss: 2.1453 - val_accuracy: 0.9139 - val_loss: 0.3915
Epoch 2/10
45/45 0s 5ms/step - accuracy: 0.9194 - loss: 0.3127 - val_accuracy: 0.9417 - val_loss: 0.1919
Epoch 3/10
45/45 0s 4ms/step - accuracy: 0.9552 - loss: 0.1668 - val_accuracy: 0.9694 - val_loss: 0.1205
Epoch 4/10
45/45 0s 4ms/step - accuracy: 0.9784 - loss: 0.0928 - val_accuracy: 0.9667 - val_loss: 0.1112
Epoch 5/10
45/45 0s 4ms/step - accuracy: 0.9906 - loss: 0.0672 - val_accuracy: 0.9833 - val_loss: 0.0751
Epoch 6/10
45/45 0s 4ms/step - accuracy: 0.9922 - loss: 0.0479 - val_accuracy: 0.9806 - val_loss: 0.0607
Epoch 7/10
45/45 0s 4ms/step - accuracy: 0.9947 - loss: 0.0325 - val_accuracy: 0.9778 - val_loss: 0.0678
Epoch 8/10
45/45 0s 4ms/step - accuracy: 0.9998 - loss: 0.0232 - val_accuracy: 0.9861 - val_loss: 0.0504
Epoch 9/10
45/45 0s 4ms/step - accuracy: 0.9969 - loss: 0.0181 - val_accuracy: 0.9806 - val_loss: 0.0502
Epoch 10/10
45/45 0s 4ms/step - accuracy: 0.9991 - loss: 0.0132 - val_accuracy: 0.9833 - val_loss: 0.0521
12/12 0s 3ms/step - accuracy: 0.9775 - loss: 0.0511
Test Accuracy: 0.9833333492279053

توضیحات مثال کد پایتون

در این مثال، یک شبکه عصبی کانولوشنی (CNN) ساده برای دسته‌بندی تصاویر ارقام از مجموعه داده **digits** طراحی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا تصاویر 8×8 از مجموعه داده digits بارگذاری شده‌اند و به دو بخش آموزش و تست تقسیم شده‌اند.
- تصاویر به شکل مناسب برای ورودی شبکه کانولوشنی (با کانال تک‌بعدی) تبدیل شده‌اند.
- مدل شامل یک لایه کانولوشن، یک لایه Pooling، و دو لایه Dense است که در نهایت خروجی softmax برای ۱۰ کلاس تولید می‌کند.
- کامپایل مدل با تابع زیان `sparse_categorical_crossentropy` و بهینه‌ساز **Adam** انجام شده و آموزش مدل روی داده‌های آموزش انجام شده است.
- در پایان، دقت مدل روی داده‌های تست محاسبه شده است.

این مثال پایه‌ای مناسب برای درک معماری‌های CNN در مسائل دسته‌بندی تصویری است و می‌تواند به راحتی برای داده‌های پیچیده‌تر مانند CIFAR یا MNIST توسعه یابد.

Section 10.4

10.4.1. Basic RNN models and applications

شبکه‌های عصبی بازگشتی (Recurrent Neural Networks - RNNs)

شبکه‌های عصبی بازگشتی (Recurrent Neural Networks - RNNs) نوعی از شبکه‌های عصبی هستند که برای مدیریت داده‌های ترتیبی (sequential data) با وارد کردن یک حلقه بازخورد (feedback loop) طراحی شده‌اند که به اطلاعات اجازه می‌دهد باقی بمانند. مدل‌های RNN پایه‌ای دارای معماری ساده‌ای هستند که آن‌ها را قادر می‌سازد وابستگی‌ها و الگوها را در توالی‌ها ثبت کنند. آن‌ها کاربردهایی در حوزه‌های مختلف از جمله پردازش زبان طبیعی، تحلیل سری‌های زمانی و تشخیص گفتار پیدا می‌کنند.

نکات کلیدی درباره مدل‌های RNN پایه‌ای و کاربردها:

- **پردازش ترتیبی:** RNN ها توالی‌ها را با حفظ وضعیت‌های پنهان (hidden states) که اطلاعات گام‌های زمانی قبلی را ثبت می‌کنند، پردازش می‌نمایند. این امر آن‌ها را قادر به مدل‌سازی وابستگی‌ها در داده‌های ترتیبی می‌سازد.
- **مسئله گرادیان ناپدیدشونده:** RNN های پایه‌ای از مسئله گرادیان ناپدیدشونده رنج می‌برند که ثبت وابستگی‌های بلندمدت را برای آن‌ها دشوار می‌سازد. این محدودیت به توسعه معماری‌های پیشرفته‌تر RNN منجر شد.
- **کاربردها:** RNN های پایه‌ای در کاربردهایی مانند مدل‌سازی زبان، ترجمه ماشینی، تحلیل احساسات و پیش‌بینی قیمت سهام استفاده می‌شوند. آن‌ها برای وظایفی که در آن‌ها زمینه عناصر قبلی در یک توالی حیاتی است، مناسب هستند.
- **بازشدن در زمان:** RNN ها را می‌توان به عنوان بازشده در طول زمان تصور کرد، به طوری که هر گام زمانی یک ورودی متفاوت در توالی را نشان می‌دهد. این بازشدن نشان می‌دهد که چگونه اطلاعات از طریق شبکه جریان می‌یابد.
- **چالش‌ها:** در حالی که RNN های پایه‌ای شهودی هستند، با چالش‌هایی در ثبت وابستگی‌های دوربرد مواجه‌اند و معماری‌های پیشرفته‌تری مانند حافظه کوتاه‌مدت طولانی (Long Short-Term Memory - LSTM) و واحد بازگشتی دروازه‌دار (Gated Recurrent Unit - GRU) برای رفع این مسائل توسعه یافته‌اند.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک RNN پایه‌ای با استفاده از TensorFlow و Keras:

In [39]:

```
# IMDB برای تحلیل احساسات روی دیتاست LSTM آموزش: ساخت مدل

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, models

# 1 با محدود کردن واژگان به ۱۰,۰۰۰ کلمه پرتکرار IMDB بارگذاری دیتاست
num_words = 10000
max_sequence_length = 100
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=num_words)

# 2 Padding یکسان‌سازی طول توالی‌ها با
X_train = pad_sequences(X_train, maxlen=max_sequence_length, padding='post')
X_test = pad_sequences(X_test, maxlen=max_sequence_length, padding='post')

# 3 برای کاهش بیش‌برازش Dropout پایه‌ای با لایه LSTM تعریف مدل
model = models.Sequential([
    layers.Embedding(input_dim=num_words, output_dim=64, input_length=max_sequence_length),
    layers.LSTM(64),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

# 4 summary ساخت مدل قبل از نمایش
model.build(input_shape=(None, max_sequence_length))
model.summary()

# 5 Adam و بهینه‌ساز binary_crossentropy کامپایل مدل با تابع زیان
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 6 آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست
history = model.fit(X_train, y_train, epochs=5, batch_size=32, validation_data=(X_test, y_test))

# 7 ارزیابی مدل روی داده‌های تست
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

Model: "sequential_31"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None , 100, 64)	640,000
lstm_2 (LSTM)	(None , 64)	33,024
dropout_8 (Dropout)	(None , 64)	0
dense_69 (Dense)	(None , 1)	65

Total params: 673,089 (2.57 MB)

Trainable params: 673,089 (2.57 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

782/782 11s 12ms/step - accuracy: 0.6960 - loss: 0.5561 - val_accuracy: 0.7030 - val_loss: 0.5578

Epoch 2/5

782/782 11s 14ms/step - accuracy: 0.8202 - loss: 0.4188 - val_accuracy: 0.8207 - val_loss: 0.4333

Epoch 3/5

782/782 18s 11ms/step - accuracy: 0.8853 - loss: 0.3060 - val_accuracy: 0.8386 - val_loss: 0.4065

Epoch 4/5

782/782 11s 11ms/step - accuracy: 0.9116 - loss: 0.2397 - val_accuracy: 0.8358 - val_loss: 0.4190

Epoch 5/5

782/782 9s 11ms/step - accuracy: 0.9272 - loss: 0.2022 - val_accuracy: 0.8472 - val_loss: 0.4053

782/782 3s 4ms/step - accuracy: 0.8470 - loss: 0.4132

Test Accuracy: 0.8472

توضیحات مثال کد پایتون

در این مثال، یک مدل **LSTM** ساده برای تحلیل احساسات روی دیتاست **IMDB** طراحی شده است. مراحل انجام شده به شرح زیر است:

- ابتدا داده‌های متنی شامل نقدهای فیلم بارگذاری شده‌اند و فقط ۱۰,۰۰۰ واژه پرتکرار نگه داشته شده‌اند.
 - توالی‌های متنی با استفاده از `pad_sequences` به طول ثابت ۱۰۰ تبدیل شده‌اند.
 - مدل شامل یک لایه **Embedding** برای تبدیل واژه‌ها به بردار، یک لایه **LSTM** برای یادگیری وابستگی‌های زمانی، و یک لایه **Dropout** برای کاهش بیش‌برازش است.
 - خروجی مدل یک مقدار احتمال برای دسته‌بندی دودویی (مثبت یا منفی) است که با تابع **sigmoid** فعال‌سازی شده است.
 - مدل با تابع زیان `binary_crossentropy` و بهینه‌ساز **Adam** آموزش دیده و روی داده‌های تست ارزیابی شده است.
- این معماری پایه‌ای مناسب برای شروع در حوزه **تحلیل احساسات و پردازش زبان طبیعی** است و می‌تواند با افزودن لایه‌های پیچیده‌تر یا استفاده از مدل‌های پیش‌آموزش‌دیده مانند BERT توسعه یابد.

10.4.2. Gated RNNs

شبکه‌های عصبی بازگشتی دروازه‌دار (Gated Recurrent Neural Networks - RNNs)

شبکه‌های عصبی بازگشتی دروازه‌دار (Gated Recurrent Neural Networks - RNNs) با معرفی مکانیزم‌های دروازه‌گذاری (gating mechanisms) که به مسئله گرادیان ناپدیدشونده (vanishing gradient problem) می‌پردازند، پیشرفتی نسبت به RNN‌های پایه‌ای محسوب می‌شوند. RNN‌های دروازه‌دار، مانند حافظه کوتاه‌مدت طولانی (Long Short-Term Memory - LSTM) و واحد بازگشتی دروازه‌دار (Gated Recurrent Unit - GRU)، برای ثبت وابستگی‌های بلندمدت در داده‌های ترتیبی به طور مؤثرتری طراحی شده‌اند.

نکات کلیدی درباره RNN‌های دروازه‌دار:

- مسئله گرادیان ناپدیدشونده:** RNN‌های دروازه‌دار مسئله گرادیان ناپدیدشونده را که در RNN‌های پایه‌ای با آن مواجه می‌شویم و یادگیری وابستگی‌های دوربرد را در توالی‌ها مختل می‌کند، کاهش می‌دهند.
- LSTM و GRU:** دو معماری محبوب RNN‌های دروازه‌دار هستند. آن‌ها مکانیزم‌های دروازه‌گذاری را برای کنترل جریان اطلاعات ترکیب می‌کنند و به شبکه اجازه می‌دهند اطلاعات را به طور انتخابی از گام‌های زمانی قبلی حفظ یا دور بیندازد.
- سلول‌های حافظه:** هم LSTM و هم GRU سلول‌های حافظه‌ای را معرفی می‌کنند که می‌توانند اطلاعات را برای دوره‌های طولانی ذخیره کنند. این سلول‌های حافظه شبکه را قادر می‌سازند اطلاعات مرتبط را در طول توالی‌های طولانی ثبت کنند.
- مکانیزم‌های دروازه‌گذاری:** مکانیزم‌های دروازه‌گذاری شامل دروازه ورودی (input gate)، دروازه فراموشی (forget gate) و دروازه خروجی (output gate) هستند. این دروازه‌ها جریان اطلاعات را تنظیم می‌کنند و این امکان را برای شبکه فراهم می‌سازند که حافظه خود را به طور انتخابی به‌روزرسانی کند.

- **کاربردها:** RNNهای دروازه‌دار کاربردهایی در وظایفی که به درک زمینه و وابستگی‌های بلندمدت نیاز دارند، مانند پردازش زبان طبیعی، تشخیص گفتار و پیش‌بینی سری‌های زمانی پیدا می‌کنند.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای یک RNN دروازه‌دار با استفاده از معماری LSTM و TensorFlow و Keras:

```
In [42]: # IMDB ساده برای تحلیل احساسات روی دیتاست LSTM آموزش: ساخت مدل

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, models

# 1 با محدود کردن واژگان به ۱۰,۰۰۰ کلمه پرتکرار Keras از IMDB بارگذاری دیتاست
num_words = 10000
max_sequence_length = 100
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=num_words)

# 2 Padding یکسان‌سازی طول توالی‌ها با
X_train = pad_sequences(X_train, maxlen=max_sequence_length, padding='post')
X_test = pad_sequences(X_test, maxlen=max_sequence_length, padding='post')

# 3 ساده با ۳۲ واحد LSTM تعریف مدل
model = models.Sequential([
    layers.Embedding(input_dim=num_words, output_dim=64, input_length=max_sequence_length),
    layers.LSTM(32),
    layers.Dense(1, activation='sigmoid')
])

# 4 Adam و بهینه‌ساز binary_crossentropy کامپایل مدل با تابع زیان
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 5 آموزش مدل روی داده‌های آموزش و اعتبارسنجی روی داده‌های تست
model.fit(X_train, y_train, epochs=5, batch_size=32, validation_data=(X_test, y_test))

# 6 ارزیابی مدل روی داده‌های تست
_, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
Epoch 1/5
782/782 ————— 11s 12ms/step - accuracy: 0.7260 - loss: 0.5283 - val_accuracy: 0.8447 - val_loss: 0.3695
Epoch 2/5
782/782 ————— 8s 10ms/step - accuracy: 0.8884 - loss: 0.2860 - val_accuracy: 0.8522 - val_loss: 0.3567
Epoch 3/5
782/782 ————— 10s 10ms/step - accuracy: 0.9186 - loss: 0.2228 - val_accuracy: 0.8411 - val_loss: 0.3703
Epoch 4/5
782/782 ————— 11s 14ms/step - accuracy: 0.9444 - loss: 0.1618 - val_accuracy: 0.8384 - val_loss: 0.5055
Epoch 5/5
782/782 ————— 18s 11ms/step - accuracy: 0.9617 - loss: 0.1140 - val_accuracy: 0.8370 - val_loss: 0.4247
782/782 ————— 3s 4ms/step - accuracy: 0.8384 - loss: 0.4287
Test Accuracy: 0.8370
```

توضیحات مثال کد پایتون

در این مثال، یک مدل **LSTM** ساده برای تحلیل احساسات روی دیتاست **IMDB** طراحی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا نقدهای متنی فیلم‌ها بارگذاری شده‌اند و فقط ۱۰,۰۰۰ واژه پرتکرار نگه داشته شده‌اند.
- توالی‌های متنی با استفاده از `pad_sequences` به طول ثابت ۱۰۰ تبدیل شده‌اند.
- مدل شامل یک لایه **Embedding** برای تبدیل واژه‌ها به بردار، یک لایه **LSTM** با ۳۲ واحد، و یک لایه خروجی **sigmoid** برای دسته‌بندی دودویی است.
- کامپایل مدل با تابع زیان `binary_crossentropy` و بهینه‌ساز **Adam** انجام شده است.
- مدل روی داده‌های آموزش آموزش دیده و روی داده‌های تست ارزیابی شده است.

این معماری پایه‌ای مناسب برای شروع در حوزه پردازش زبان طبیعی و تحلیل احساسات است و می‌تواند با افزودن لایه‌های پیچیده‌تر یا استفاده از مدل‌های پیش‌آموزش‌دیده مانند BERT توسعه یابد.

🌐 شبکه‌های عصبی کانولوشنال گراف (Graph Convolutional Networks - GCNs)

شبکه‌های عصبی کانولوشنال گراف (Graph Convolutional Networks - GCNs) نوعی از شبکه‌های عصبی گراف (Graph Neural Network) هستند که برای کار بر روی داده‌های با ساختار گراف طراحی شده‌اند. آن‌ها مفهوم شبکه عصبی کانولوشنال را به حوزه‌های گراف گسترش می‌دهند و یادگیری نمایش‌های گره (node representations) را ممکن می‌سازند که اطلاعات ساختاری محلی و کلی را ثبت می‌کنند.

نکات کلیدی درباره شبکه‌های عصبی کانولوشنال گراف:

1. **ساختار گراف:** GCN ها برای کار با داده‌هایی که به شکل یک گراف سازماندهی شده‌اند، طراحی شده‌اند؛ جایی که گره‌ها (nodes) موجودیت‌ها و یال‌ها (edges) روابط بین آن‌ها را نشان می‌دهند.
2. **نمایش‌های گره:** GCN ها بردار تعبیه‌هایی (embeddings) را برای گره‌ها در گراف یاد می‌گیرند که اطلاعات مربوط به خود گره و همسایگی آن را ثبت می‌کند.
3. **عملیات کانولوشن گراف:** عملیات کلیدی در GCN ها، کانولوشن گراف است که شامل جمع‌آوری اطلاعات از گره‌های همسایه می‌باشد. این امر به گره‌ها اجازه می‌دهد اطلاعات را از زمینه محلی خود ترکیب کنند.
4. **انتشار عمقی:** GCN ها را می‌توان در لایه‌های متعدد روی هم قرار داد تا امکان انتشار اطلاعات از طریق گراف فراهم شود. هر لایه با در نظر گرفتن زمینه‌های به طور فزاینده گسترده‌تر، نمایش‌های گره را بهبود می‌بخشد.
5. **کاربردها:** GCN ها کاربردهایی در وظایفی مانند طبقه‌بندی گره (node classification)، پیش‌بینی پیوند (link prediction) و وظایف در سطح گراف پیدا می‌کنند. آن‌ها در سناریوهایی که درک روابط و وابستگی‌ها در یک گراف ضروری است، مفید هستند.

اکنون، بیا یک مثال عملی در پایتون (Python) برای یک شبکه عصبی کانولوشنال گراف با استفاده از کتابخانه PyTorch Geometric (PyG) ببینیم و همچنین نمایش تصویری embeddings گره‌ها را مشاهده کنیم تا جریان اطلاعات در گراف ملموس شود:

```
In [7]: !pip install --quiet torch-scatter torch-sparse torch-cluster torch-spline-conv torch-geometric
```

```
In [3]: # Cora ساده برای دسته‌بندی گره‌ها در گراف GCN آموزش: ساخت مدل
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.datasets import Planetoid
from torch_geometric.nn import GCNConv
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# 1 بارگذاری دیتاست از PyTorch Geometric
dataset = Planetoid(root='/tmp/Cora', name='Cora')
data = dataset[0]

# 2 با دو لایه کانولوشن گرافی GCN تعریف معماری مدل
class GCN(nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super(GCN, self).__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, out_channels)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = self.conv2(x, edge_index)
        return x

# 3 استخراج ویژگی‌ها و ماسک‌های آموزش/تست
x = data.x
edge_index = data.edge_index
```

```

y = data.y
train_mask = data.train_mask
test_mask = data.test_mask

# 4 ساخت مدل و تعریف بهینه‌ساز
model = GCN(dataset.num_node_features, 16, dataset.num_classes)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# 5 حلقه آموزش مدل به مدت ۳۰ دوره
for epoch in range(30):
    model.train()
    optimizer.zero_grad()
    out = model(x, edge_index)
    loss = F.cross_entropy(out[train_mask], y[train_mask])
    loss.backward()
    optimizer.step()
    print(f'Epoch {epoch+1}/30 | Loss: {loss.item():.4f}')

# 6 ارزیابی مدل روی داده‌های تست
model.eval()
with torch.no_grad():
    out = model(x, edge_index)
    pred = out.argmax(dim=1)
    accuracy = (pred[test_mask] == y[test_mask]).float().mean().item()
print(f'\nTest Accuracy: {accuracy:.4f}')

# 7 TSNE گره‌ها با کاهش بعد embedding نمایش تصویری
with torch.no_grad():
    hidden_emb = model.conv1(x, edge_index)
    hidden_emb = F.relu(hidden_emb)
    tsne = TSNE(n_components=2, random_state=42)
    node_pos_2d = tsne.fit_transform(hidden_emb.numpy())
    plt.figure(figsize=(8,6))
    scatter = plt.scatter(node_pos_2d[:,0], node_pos_2d[:,1], c=y.numpy(), cmap='tab10', s=40)
    plt.title("GCN Node Embeddings Visualization (Cora)")
    plt.xlabel("TSNE Component 1")
    plt.ylabel("TSNE Component 2")
    plt.legend(*scatter.legend_elements(), title="Classes")
    plt.show()

```

```

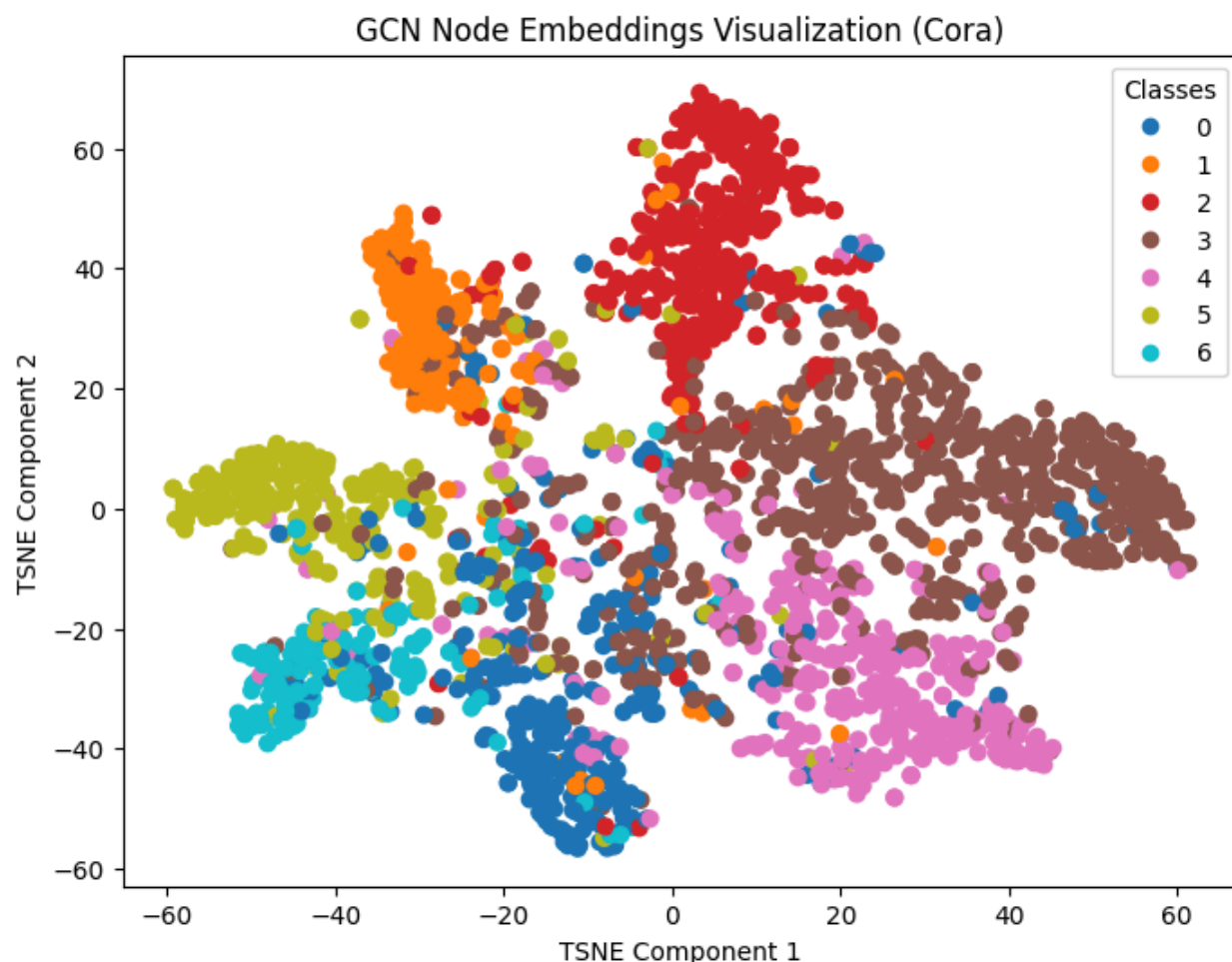
Epoch 1/30 | Loss: 1.9537
Epoch 2/30 | Loss: 1.8587
Epoch 3/30 | Loss: 1.7304
Epoch 4/30 | Loss: 1.5847
Epoch 5/30 | Loss: 1.4379
Epoch 6/30 | Loss: 1.2897
Epoch 7/30 | Loss: 1.1423
Epoch 8/30 | Loss: 1.0011
Epoch 9/30 | Loss: 0.8708
Epoch 10/30 | Loss: 0.7522
Epoch 11/30 | Loss: 0.6450
Epoch 12/30 | Loss: 0.5492
Epoch 13/30 | Loss: 0.4647
Epoch 14/30 | Loss: 0.3915
Epoch 15/30 | Loss: 0.3287
Epoch 16/30 | Loss: 0.2753
Epoch 17/30 | Loss: 0.2303
Epoch 18/30 | Loss: 0.1925
Epoch 19/30 | Loss: 0.1609
Epoch 20/30 | Loss: 0.1347
Epoch 21/30 | Loss: 0.1128
Epoch 22/30 | Loss: 0.0947
Epoch 23/30 | Loss: 0.0798
Epoch 24/30 | Loss: 0.0674
Epoch 25/30 | Loss: 0.0572
Epoch 26/30 | Loss: 0.0487
Epoch 27/30 | Loss: 0.0417
Epoch 28/30 | Loss: 0.0359
Epoch 29/30 | Loss: 0.0311
Epoch 30/30 | Loss: 0.0271

```

```

Test Accuracy: 0.7940

```

توضیحات مثال کد پایتون

در این مثال، یک مدل **GCN** ساده برای دسته‌بندی گره‌ها در گراف **Cora** طراحی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های گرافی از مجموعه داده **Cora** بارگذاری شده‌اند که شامل ویژگی‌های گره‌ها، یال‌ها و برچسب‌هاست.
- مدل شامل دو لایه **GCNConv** است که با تابع فعال‌سازی **ReLU** بین آن‌ها ترکیب شده‌اند.
- مدل با تابع زیان **cross_entropy** و بهینه‌ساز **Adam** آموزش دیده و روی داده‌های تست ارزیابی شده است.
- در پایان، embedding گره‌ها از لایه اول استخراج شده و با استفاده از **TSNE** به فضای دوبعدی کاهش یافته‌اند.
- نمایش تصویری گره‌ها با رنگ‌بندی بر اساس کلاس‌ها انجام شده تا توزیع ویژگی‌ها در فضای یادگیری قابل مشاهده باشد.

این مثال پایه‌ای مناسب برای درک معماری‌های **گراف کانولوشن** و نحوه یادگیری نمایش گره‌ها در شبکه‌های گرافی است. می‌توان آن را به راحتی برای داده‌های پیچیده‌تر یا مدل‌های پیشرفته‌تر مانند GAT یا GraphSAGE توسعه داد.

The End of Chapter 10